Recordings, Not Rows: Lossless Waveform Storage and Lag-Tolerant Retrieval Inside PostgreSQL

Working draft 0.7 — not for distribution

Zin Nam

ABSTRACT

Analysts of waveform archives look for intervals that resemble a query up to a small time shift, and then need the original samples of each hit. We define this as a top-k search under a bounded-lag cross-correlation scorer and ask how it can use a standard vector index without losing the link to the recording, the semantics of metadata filters, or transactional updates. Zero-lag vector search ranks candidates by a different quantity: on 235,000 seismic windows a zero-lag shortlist of 1000 recovered only 74% of the scorer's top ten, whether taken exactly or from an HNSW index, so the loss is caused by the candidate metric, not by approximate search. Each lag term is an inner product with a zero-padded, shifted copy of the query; we therefore send a sparse grid of shifted probes (13, spacing five lags, chosen empirically) to one unmodified inner-product HNSW index over unshifted stored vectors and re-rank the union with the unchanged scorer. Query-side shifting and multi-query candidate generation are known; we contribute their combination for this scorer, its integration in PostgreSQL, and its evaluation. On stations and a year held out from development, this recovered 97.6% of the top ten (per query at least 70%) at a p95 latency of 131 ms, against 713 ms for the fastest exact path we measured; zero-lag candidates of the same number recovered 61%. MorletDB, a PostgreSQL extension, implements candidate generation, exact re-ranking and the return of hits as sample intervals of losslessly stored recordings. The speed-up has a price: the indexed copy added 1.0–1.4× the raw storage; selective filters required routing small galleries to exact search, without which recall fell to 0.52; and maintaining the index at commit multiplied write-ahead logging unless checkpoint settings were changed.

1 INTRODUCTION

Seismic, vibration and physiological archives consist of long, uniformly sampled recordings. A common investigation starts from one interval and asks where similar behaviour occurred, then inspects the original samples and acquisition context of each candidate. Two practical requirements follow. Similarity must tolerate a small time shift, because a window grid rarely aligns two occurrences of the same event; and a result is useful only if it identifies an interval of a recording that can be read back exactly.

A bounded-lag cross-correlation captures the first requirement: two windows are similar if some shift of at most L samples correlates them strongly (Section 2). Exhaustive evaluation is linear in the archive. The obvious acceleration is a vector index, which PostgreSQL users already have through pgvector [1]; but such indexes rank by a fixed metric between unshifted vectors. This paper asks:

> *How can bounded-lag waveform search be accelerated with standard vector indexes while preserving the link to the recording, the semantics of filters and transactional updates, and which costs limit its use?*

Our argument has six steps. (1) Waveform investigations need intervals similar under a bounded shift, with access to their original samples. (2) Zero-lag vector search optimizes a different candidate ranking and loses neighbours of the defined scorer. (3) A diagnostic on the same data separates this effect from approximate search: an exact and an HNSW zero-lag shortlist lose the same neighbours (Section 6). (4) Shifted inner-product probes target exactly this difference: each lag term of the scorer is an inner product with a shifted query, so a few shifted queries against one index generate candidates for the scorer actually used (Section 3). (5) MorletDB places candidate generation, exact re-ranking and sample-interval results in PostgreSQL, next to losslessly stored recordings (Section 4). (6) Filters, index storage and maintenance cost determine the range in which this is worthwhile (Sections 8–10).

Contributions.

(1) **A specific candidate-generation method for a bounded-lag scorer:** query-side, zero-padded shifted probes on a sparse lag grid sent to an unmodified inner-product ANN index, a union of their candidates, and re-ranking with the unchanged scorer. Its ingredients are known (Section 11); the combination needs neither a trained embedding, nor per-shift copies of the data, nor lower bounds, at the price of approximate candidate selection (Section 3).
(2) **A recording-oriented integration in PostgreSQL:** results are sample intervals of losslessly stored recordings; search representations and their index rows are maintained in the append transaction; eligibility filters and row-level security apply to both paths; a router chooses exact or probe search by the number of eligible windows (Section 4).
(3) **A controlled evaluation of benefit and limits:** a diagnosis of why zero-lag shortlists fail, an ablation that separates lag-grid sparsity from approximate search, confirmation on held-out stations and year, and measured costs of filters, storage and index maintenance (Sections 6–10).

Section 11 compares these with the closest prior work and states what we consider new. We do not claim a new similarity measure, a new index, lossless compression, or guarantees for probe search: probe search is approximate in the candidates it considers, and its recall is measured, not bounded. Recall in this paper is fidelity to the scorer, not relevance to a domain question. Code, frozen protocols and raw results are public [5]; a supplement reports storage, recovery and other studies not needed for the argument here.

2 PROBLEM AND SEARCH CONTRACT

A *recording* is a sequence $x_0, \ldots, x_{N-1}$ of float64 samples with a sample rate, a channel, a unit and JSON acquisition metadata. An *interval* $[a, b)$ denotes $x_a, \ldots, x_{b-1}$; sample offsets, not wall-clock times, are authoritative. A *window* is an interval of fixed length w on a stride grid. A search representation $\phi(x_{a:b}) \in \mathbb{R}^n$ is derived from a window by a versioned extractor.

We study envelopes: for $w = 4096$, z-score the window, take the magnitude of the analytic signal, smooth with a 50-sample moving mean, keep every tenth value and z-score again ($n = 410$). Stored envelopes are float16. For a query envelope q computed at full precision and a stored envelope g, the scorer is

$$D(q, g) \;=\; 1 - \max_{|\ell| \le L} \frac{1}{n} \sum_t q_{t+\ell}\, g_t, \qquad L = \lfloor 0.075\, n \rfloor = 30, \quad (1)$$

where terms with $t + \ell$ outside $[0, n)$ are zero. This is a *biased* cross-correlation: no per-lag renormalization of the overlapping part, no circular shift, and one z-score per window rather than per overlapping subsequence. At 100 Hz, $L = 30$ envelope values correspond to ±3 s.

Contract. A query names a source interval (or supplies samples), a metadata filter and k. The *eligible* windows are those whose recordings have the query's sample rate, channel and unit, whose window size and extractor match, whose metadata contain the filter, and which the caller may see under row-level security. The *exact result* is the first k eligible windows in the order $(D, \text{recording}, \text{start})$, after excluding windows overlapping the source interval and greedily skipping windows that overlap a better window of the same recording. Each result row is a recording identifier, an interval and its distance; the interval reads back the original samples bit-exactly from the raw store.

Three kinds of exactness. We keep apart (i) *exact distances*: every returned distance is D computed on the stored envelope; (ii) *exact candidate selection*: the returned set equals the exact result; and (iii) *relevance*: the returned intervals help answer the analyst's question. Probe search guarantees (i) but not (ii); we measure (ii) as recall@10 against the exhaustive search; neither implies (iii).

3 SHIFTED INNER-PRODUCT PROBES

For a lag ℓ, define the zero-padded shifted query $s_\ell[t] = q_{t+\ell}$ if $0 \le t + \ell < n$ and 0 otherwise. Then $\sum_t q_{t+\ell} g_t = \langle s_\ell, g \rangle$ and

$$1 - D(q, g) \;=\; \frac{1}{n} \max_{|\ell| \le L} \langle s_\ell, g \rangle. \quad (2)$$

The bounded-lag top-k is therefore the top-k of a maximum over $2L{+}1$ inner-product queries against the *unshifted* stored vectors. Two consequences matter. The shift is applied to the query, so the index holds one copy of each window, not one per lag. And the probes must be inner-product queries, not cosine: $\|s_\ell\|$ shrinks with $|\ell|$ because of zero padding, exactly as the biased scorer weights a shifted overlap.

Algorithm 1 replaces the full lag set by a sparse grid ($|\Lambda| = 13$ at spacing 5) and each exact inner-product top-c by an HNSW search. Three approximations are thus stacked, and the evaluation separates them: (a) *grid sparsity*: a window whose best lag lies between grid points must still rank in the top c of a neighbouring probe, which holds when envelopes are smooth relative to the spacing, but is not guaranteed; 13 probes at spacing five are an empirically chosen configuration for these envelopes, not a general setting; (b) *per-probe budget*: a window that is the global top ten under D need not be in any single probe's top c; (c) *index approximation*: HNSW returns an approximate top-c, and it compares a float16 copy of the probe with float16 stored vectors, whereas Equation 2 holds for the full-precision query. Re-ranking uses the full-precision query and the stored envelope, so (a)–(c) affect only which windows are considered, never a returned distance. Properties of D, such as tolerance to shifts within $\pm L$, do not transfer to the quantized index probes without measurement; Section 7 measures them.

Algorithm 1 Probe search (product function, simplified)

Require: query envelope q (full precision), eligibility predicate E, k; grid $\Lambda = \{-L, -L{+}5, \ldots, L\}$; per-probe budget $c = 50$; threshold $T = 5000$

```
if |{g : E(g)}| ≤ T then ▷ counted from recordings, stopping at T
    return ExactScan(q, E, k)
end if
C ← ∅
for ℓ ∈ Λ do
    C ← C ∪ top-c by ⟨f16(s_ℓ), g⟩ over g with E(g)    ▷ HNSW, iterative scan
end for
score every g ∈ C with D(q, g) on its stored envelope
return first k in order (D, rec, start) after source exclusion and overlap suppression
```

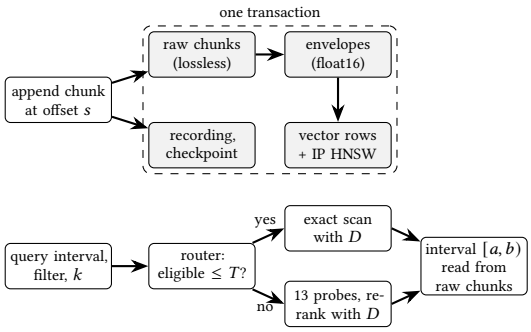


Figure 1: Write path and the two search paths. Raw chunks, envelopes, vector rows and the checkpoint commit together. Both search paths score candidates with D on stored envelopes and return sample intervals, which are read from the raw chunks, not from a representation. The exact path reads envelopes; the probe path reads vector rows for candidates and envelopes for re-ranking.

4 INTEGRATION IN POSTGRESQL

MorletDB is a PostgreSQL 17 extension written in Rust with pgrx. Figure 1 shows the paths that matter for search; the supplement describes storage.

Recordings and appends. A collection is a set of caller-owned tables: recordings with a checkpoint, raw chunks, envelopes and,

optionally, a pgvector table of `halfvec` rows with an inner-product HNSW index. An append names its global start offset; raw chunk, new envelopes, their vector rows and the checkpoint commit in one transaction, so a window is searchable exactly when the transaction that completes it commits, and a committed checkpoint never refers to uncommitted samples. A repeated identical append is a verified no-op; a gap, overlap or changed retry fails. Raw chunks are coded losslessly (polynomial prediction, Rice coding and a float-to-integer grid with exact corrections; supplement), so a result interval reads back the appended float64 bit patterns. Envelopes and vector rows are derived and can be dropped and rebuilt without touching raw data.

Two search paths, one contract. The exact path scores every eligible stored envelope with D in one parallel SQL statement. The probe path implements Algorithm 1; both apply the same eligibility, exclusion, ordering and suppression, and both return distances computed by the same Rust scorer. All functions run as `SECURITYINVOKER`, so privileges and row-level security apply to every table read.

Filters. An approximate index answers "nearest overall"; a filter asks "nearest among eligible". Applying eligibility after probing silently discards candidates when few windows qualify. The router therefore counts eligible windows from the recordings table, stopping at the threshold T, and sends small galleries to the exact path; above T, each probe scans only eligible recordings, using pgvector's iterative index scan. Choosing between brute force and an index by filter selectivity is established practice in vector databases (Section 11); we include it because it decides whether probe search keeps the contract under filters.

Access control. Row-level security on the recordings table protects search functions but not direct reads of the derived tables: a first deployment test found that another tenant's chunks, envelopes and vector rows were readable directly. `morlet_enable_row_security` now attaches a policy to every companion table that admits rows whose recording is visible; written as membership in an array of visible identifiers it kept parallel scans (exact search +5%), whereas a correlated `EXISTS` tripled exact search time.

5 EVALUATION SETUP

Protocols. Each study committed its questions, data, arms, metrics and decision thresholds before reading or measuring data, ran once, and logged deviations [5]. Table 1 lists the studies used here.

Data. Development data (*dev*) are CX HHZ recordings [4] of stations PB01, PB02, PB06 and PSGCX, 1–14 March 2025: 481 M samples at 100 Hz, 234,762 windows of 4096 samples at stride 2048. Held-out data (*held-out*) are stations PB05, PB11, PB14 and PB18, 1–14 March 2021 (234,622 windows), chosen from the data centre's availability listing before any waveform was downloaded; no earlier study had read them. Each set has 100 queries from its first day: 50 random and 50 high-energy windows. Seismic envelopes are one signal domain; we discuss generality in Section 12.

Measurements. We separate three kinds: *experiment* (harness code that issues SQL arms and scores offline controls), *product* (the released SQL functions on the same queries), and *historic* (an earlier implementation, reported where it explains a design decision). Recall@10 is against the exhaustive exact result. All runs used one

Table 1: Studies, data and measurement kind.

Study	Question	Data
R2, R2-D	exact scaling; zero-lag shortlists, exact vs HNSW	dev
BL1	probe grids and budgets, ablation controls	dev
BL1-C	frozen configuration, once	held-out
SS3	filtered search and router; product check	dev, held-out
SS1, R1b	one stored copy; float16 precision	dev, held-out; 2022
BL1-S, C2	index maintenance during ingestion	dev; dev+held-out

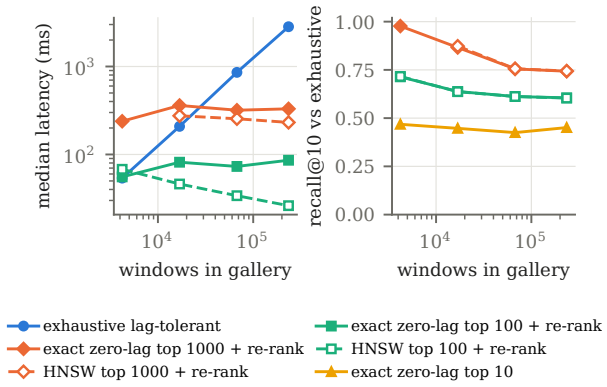


Figure 2: R2, R2-D on dev galleries. Left: median latency of the exhaustive scan (historic, single-threaded implementation) and of zero-lag shortlists plus re-ranking (exact: solid; HNSW: dashed). Right: recall@10; the HNSW curve lies on the exact one. 100 queries per gallery.

Apple M5 Max (128 GB), PostgreSQL 17.11, pgvector 0.8.0 (HNSW m=16, `ef_construction` 64, `ef_search` 100), warm caches and, except Section 10, one client. During the later studies an unrelated CPU-intensive job shared the machine; we report its load average where it changed between compared runs.

6 WHY ZERO-LAG SHORTLISTS FAIL

A natural first stage takes the top K windows by zero-lag cosine from an ANN index and re-ranks them with D. R2 measured this on dev galleries nested from 4,217 to 234,762 windows [7]. R2's statement passed its query vector through a subquery, which pgvector cannot use as an index ordering key, so it silently computed an *exact* zero-lag top K. A diagnostic on the same data (R2-D, protocol frozen before measuring) therefore compared an explicitly exact shortlist with an HNSW shortlist whose plan was checked to use the index.

With K=1000, recall@10 fell from 0.977 on the smallest gallery to 0.743 on the largest; with K=100, from 0.714 to 0.605 (Figure 2). The HNSW shortlist agreed with the exact one on 98–100% of candidates on the largest gallery and gave the same recall to within 0.007 at every size (0.744 at K=1000). Every top-ten item contained

Table 2: Recall@10 by candidate generator (13 probes × 50 unless noted), 100 queries per set: mean / minimum / queries below 0.9. All arms re-rank with D; union sizes 338–369 windows on average. "—": not run.

Candidates	dev	held-out
all 61 lags, exact (PX-61)	1.000 / 1.0 / 0	—
13-lag grid, exact (PX)	1.000 / 1.0 / 0	0.999 / 0.9 / 0
7-lag grid, exact	0.986 / 0.7 / 3	—
13-lag grid, f16, HNSW	**0.998 / 0.9 / 0**	**0.976 / 0.7 / 5**
7-lag grid, f16, HNSW	0.984 / 0.7 / 3	—
zero-lag, exact, same count (EZ)	0.681 / 0.0 / 66	0.613 / 0.0 / 69

in a shortlist survived re-ranking and suppression, so coverage equalled recall. The loss is therefore caused by the zero-lag candidate metric, not by approximate search or by the contract's exclusion rules: a quarter of the bounded-lag top ten lies outside the 1000 windows nearest by zero-lag cosine. The exact zero-lag top ten alone shared 43–47% of the bounded-lag top ten. These are two budgets and one HNSW configuration. The exhaustive scan in Figure 2 is the historic implementation (linear, 12.1–12.8 µs per window); the current exact path runs as one parallel statement and is the baseline below.

7 SHIFTED PROBES: ABLATION AND CONFIRMATION

BL1 fixed grids of 7 and 13 probes and budgets of 50 and 100 per probe, with offline controls computed in the harness from the same stored float16 envelopes and the full-precision query: *PX*, the same probe union taken *exactly* (no index, no probe quantization); *PX-61*, all 61 lags exactly; and *EZ*, an exact zero-lag cosine shortlist with as many candidates as the probe union [10]. The gate required recall@10 ≥ 0.95, p95 at most a third of the exhaustive scan and below the fastest exact path, and added storage at most 1.5× raw. All four configurations passed on dev; 13 probes × 50 was frozen and run once on held-out data.

Ablation. Table 2 separates the approximations of Section 3. On dev, the 13-lag grid taken exactly lost nothing against all 61 lags, while the 7-lag grid lost 0.014, so sparsity at spacing 5 cost nothing measurable and at spacing 10 did. Index approximation, including float16 probes, cost 0.002 on dev and 0.023 on held-out data, where five queries fell below 0.9 and the worst reached 0.7; the losses concentrated on high-energy queries (0.954 against 0.998 for random ones). With the same number of candidates, the zero-lag shortlist reached 0.61–0.68 and failed two thirds of queries.

Why spacing five suffices here. A window whose best lag lies d lags from the nearest probe keeps, in that probe, roughly the envelope's own correlation at offset d. On 3,000 random dev envelopes this biased autocorrelation was 0.973 at d=2 (5th percentile 0.944) but 0.895 at d=5 (5th percentile 0.785) [33]. With spacing five no top-ten item is more than two lags from a probe; with spacing ten 31% of dev top-ten items lie three to five lags away, where 5% of envelopes lose more than a fifth of their correlation. The spacing is thus tied to the representation's smoothing (a 50-sample moving mean before decimation by ten, about five envelope values); a less

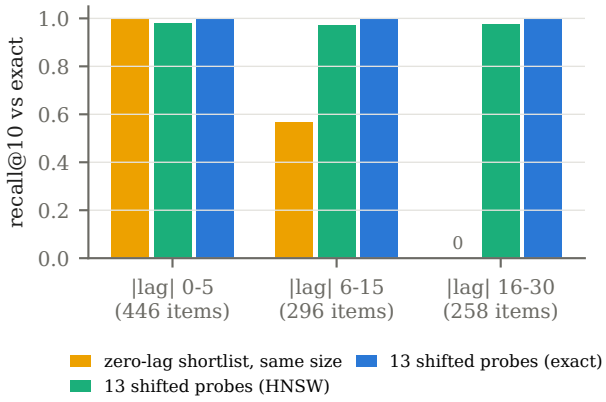


Figure 3: Held-out: recall of the bounded-lag top ten by the lag at which each item scores best. Zero-lag candidates of the same number find aligned matches but none beyond ±15; probes are flat across lags.

Table 3: Held-out confirmation, unfiltered, 234,622 windows, 100 queries. Exhaustive: historic implementation.

	Exhaustive	Parallel exact	**Probes**	Product
Kind	historic	experiment	**experiment**	product
Recall@10	1	1	**0.976**	0.977
p50 / p95 (ms)	2,834 / 2,854	698 / 713	**97 / 131**	— / 50
Windows re-ranked	234,622	234,622	357	—
Added storage / raw	—	—	1.00	1.00

smooth representation would need a denser grid, which we have not measured.

Figure 3 shows the mechanism. Of the held-out top-ten items, 55% score best at a lag beyond ±5. Zero-lag candidates recovered 0.998 of items with best lag within ±5, 0.568 between 6 and 15, and none beyond 15; probes recovered 0.973–0.978 in each band.

Confirmation and latency. All confirmation criteria held (Table 3). Against the fastest exact path measured on the same queries, a parallel SQL statement, the probe arm was 5.4× faster at p95. The product function, whose probe loop runs inside the server, reached 0.977 at 50 ms on the same queries; it is a later implementation and not part of the frozen comparison. A first product version used a smaller HNSW search breadth than the evaluated configuration and reached 0.963; it was aligned with the evaluated setting.

8 FILTERS AND ROUTING

Investigations usually restrict the gallery to one sensor or period. The first probe implementation applied eligibility after probing. SS3 compared three strategies at five filter levels per query [12]: exact search, post-filtered probes and filter-aware probes; a router threshold was fixed on dev data before the held-out run.

Post-filtering lost up to half the top ten (Table 4). Filter-aware probes restored recall but were slow at the smallest level (p95 724 ms), where the planner abandoned the index and exact search

Table 4: Filtered search on held-out data, mean recall@10 (product: also minimum) against exact search, 100 queries per level. Post-f.: post-filtered probes; F-aware: filter-aware probes; Routed: product function (*routed to exact).

Eligible	Exact p95	Post-f.	F-aware	Routed mean/min	Routed p95
1.8% (4,217)	68 ms	0.515	0.998	**1.0**/1.0*	85 ms
7.2% (16,868)	329 ms	0.848	0.997	**0.997**/0.9	207 ms
25% (58.6 k)	290 ms	0.972	0.986	**0.986**/0.6	98 ms
29% (67.5 k)	390 ms	0.976	0.994	**0.994**/0.8	88 ms
100% (234.6 k)	~1.2 s	0.976	0.977	**0.977**/0.7	69 ms

took 68 ms. The router sends that level to exact search. No returned row violated its filter, and with row-level security restricting a role to one station, every strategy returned only that station and rejected a hidden recording as query source. The router missed its pre-registered latency bound once, at the unfiltered held-out level, by 0.8 ms. Low per-query minima at 25% and 100% coincide with the high-energy queries of Section 7, not with routing errors.

9 STORAGE, RECALL AND LATENCY

Probe search costs space. On dev, raw chunks took 371 MB, envelopes 223 MB (used by both paths for scoring), the `halfvec` copy 241 MB and its HNSW index 275 MB: the index structures added 1.39× the raw data; on held-out data, whose raw data compress less well, 1.00×. Envelopes alone were 12.9% of raw on event windows at stride 4096 but 60.2% on continuous recordings at stride 2048; the factor of 4.7 is explained by stride (2.0), raw compressibility (2.46) and row overhead (0.95) [7].

Float16 storage is not the bottleneck: on 2022 windows that no earlier study had read, float16 envelopes kept recall@10 at 0.9999 against envelopes recomputed from raw samples [6]. The two stored copies hold the same float16 values in 98.3% of positions. SS1 tested keeping one [11]: a single `halfvec` table was exact on both databases (identical distances for 9.4 M pairs) and saved about 250 MB per 235,000 windows, but made exact scans 23% slower; an HNSW index on an expression over the envelope table kept exact search unchanged but raised probe p95 by 27–32%. Both missed their pre-registered latency bounds. The trade-off is therefore three-way: about 1–1.4× raw storage bought a 5.4× lower p95 than the fastest exact path in the frozen comparison, at recall 0.976; saving a third of that storage costs 20–30% latency on one path.

10 MAINTENANCE DURING INGESTION

Maintaining vector rows at commit makes new windows searchable immediately, but every append now updates an HNSW graph. Two measurements bound this cost; neither is a long-term stability result.

Replay (BL1-S, dev, one day of four channels, 16,868 windows). With maintenance in the append transaction, every window was searchable at its commit while appends ran at 919,000 samples/s, 53% of raw-only throughput. At 200× real time, probe queries during ingestion had a p95 of 82 ms against 189 ms for exact search, at recall 0.9997 on the same snapshot [10].

Sustained load (15-minute runs). On an archive of 938,768 windows (dev and held-out recordings, each also replayed time-reversed), four writers appended 16 channels at 25.6 kHz (409,600 samples/s, a vibration-monitoring rate) with maintenance in the append transaction, while four clients issued one query every two seconds (half filtered, half unfiltered) [13]. With PostgreSQL's default configuration the run met its pre-registered criteria (filtered p95 167 ms; probe p95 337 ms, spot-check recall 0.998 against exact search in the same snapshot; ingest lag at most 22 s and recovered), but wrote 13.8 GB of write-ahead log for 0.76 GB of database growth. At twice the rate, the writers fell behind without recovering; capacity lay between 1.3 and 1.4× the target. A second 15-minute run changed four settings together (shared buffers 128 MB→8 GB, `max_wal_size` 1→16 GB, checkpoint timeout 5→15 min, WAL compression on) and wrote 1.91 GB of WAL for 0.65 GB of growth, with probe p95 100 ms (recall 0.994) and ingest lag at most 2 s. Because the settings changed together, we cannot attribute the reduction to any one of them; plausibly, fewer checkpoints force fewer full-page images of index pages touched by HNSW insertions, but we did not isolate this. The background load average was 11.7 in the first run and 7.3 in the second, so the latency difference is confounded; the WAL volume is less sensitive to it. A two-hour run at the target rate was in progress when this draft was written.

11 RELATED WORK AND WHAT IS NEW

We looked for the strongest overlapping prior work, reading primary sources; Table 5 records for each work whether we read the full text (F), parts (P) or only the abstract (A). The full matrix, with quotations and what we searched for, is in the repository [5]. We use *shift* for a displacement along the time axis. Invariance to a vertical offset (adding a constant), as in normalization-based methods [30], and shift-invariant kernels that average over all circular shifts [19] are different notions.

What is known. Three ideas we use are established. (i) *Shifting the query instead of the data.* Rafiei [14] evaluates similarity under a set of transformations, explicitly including time shifts, against one index of untransformed sequences, and processes all transformations in one index traversal; Keogh et al. [15] expand a query into all its rotations, keep the database stored once, support rotation-limited queries by dropping rows of that matrix, and merge neighbouring rotations into a few envelopes, an exact counterpart of our sparse grid. Foulds et al. [17] search under cyclic permutations of the query. All three are exact, use Euclidean-type distances, and treat shifts as circular. (ii) *Several vector queries against one approximate index, union, exact re-ranking.* ColBERT's end-to-end retrieval issues one ANN search per query vector, takes the union of document identifiers and re-ranks it exhaustively [24]. (iii) *ANN candidates for cross-correlation.* In seismology, Tibi et al. [22] project waveforms with kernel PCA so that correlation maps to Euclidean distance, search a KD-tree forest and evaluate exact correlation on the returned set (25× faster than linear search at recall above 80%); Gonzales et al. [23] issue several internal

Table 5: Closest prior work. Shift: which displacements along the time axis the similarity tolerates. Side: where the shift is applied. Cands.: whether the candidate set is exact for the stated similarity. Read: F full text, P parts, A abstract. [a]exact for the magnitude feature, which itself approximates the rotation distance.

Work	Similarity	Shift	Side	Index	Cands.	Copies, refinement	Read
Rafiei 1999 [14]	Euclidean on normal form, truncated DFT	set of transformations incl. time shifts (circular in DFT)	transformations applied to index entries	R-tree on DFT coefficients, one traversal for all	exact	one copy; false positives removed exactly	F
Keogh et al. 2006/09 [15]	Euclidean (also DTW)	all circular rotations or a limited range	query: matrix of rotations, merged into wedges	lower bounds; VP-tree	exact	one copy; exact refinement	F
Vlachos et al. 2005 [16]	Euclidean on Fourier magnitudes	all circular	data: invariant feature	VP-tree variant	exact[a]	one compressed signature	P
Foulds et al. 2015 [17]	distance over cyclic permutations of the query	all cyclic	query	none; parallel DFT-based search	exact	—	A
k-Shape, TODS 2017 [8, 18]	normalized CC incl. biased variant	all linear shifts (FFT)	pairwise	none	exact	—	P
GRAIL 2019 [19]	kernel over all circular shifts of NCC	all circular	data: learned embedding	any (Euclidean/IP)	approx.	one embedding per series	P
Qiu et al. 2018 [20]	Pearson at zero lag	none	data: learned embedding	Euclidean index	approx.	one embedding	P
MASS, UCR suite [3, 9]	z-normalized ED per subsequence	every offset	scan	none	exact	—	A
FAST, Rong et al. [2, 21]	Jaccard on binary fingerprints	data windows every 1 s	data: dense windows	MinHash LSH	approx.	one fingerprint per overlapping window	F
Tibi et al. 2017 [22, 23]	max-over-lag CC of waveforms	bounded lags	data: kernel-PCA embedding of CC	KD-tree forest	approx.	projected vectors; exact CC on candidates	F
ColBERT 2020 [24]	sum of max inner products	—	query: one vector per token	one ANN index, one search per vector	approx.	union of candidates re-ranked exactly	F
This paper	biased, zero-padded CC	linear, $\|\ell\| \le L$	query: sparse grid of shifted copies	unmodified IP HNSW, one search per probe	approx.	one vector copy + index; union re-ranked with D	—

queries to compensate for projection error. FAST obtains shift tolerance from densely overlapping data windows [2, 21], and GRAIL learns embeddings of a shift-invariant correlation kernel [19]. The identity between a lag term and an inner product with a shifted signal is textbook; the biased statistic itself appears as NCC_b in the k-Shape work [18]. On the database side, choosing brute force over an index when a filter is selective is standard in vector systems (AnalyticDB-V, Milvus, ACORN, and the documented thresholds of Weaviate, Qdrant, Vespa and Lucene) [25–27]; VBase and pgvector's iterative scans provide filtered index scans inside PostgreSQL [1, 28]; SingleStore-V makes vector search visible at commit by indexing immutable segments and scanning fresh rows exactly [29]; and KV-match and ULISSE return exact subsequence matches as offsets in stored series [30, 31].

What is new here. We claim neither query-side shifting, nor multi-query candidate generation, nor routing as new. What we did not find in the works above, which is not evidence that it does not exist, is their combination for a *bounded, zero-padded, linear* lag scorer on an *unmodified, approximate inner-product index*: Equation 2 lets a bounded-lag score use any inner-product ANN index without training an embedding (unlike Tibi et al. and GRAIL), without replicating windows per shift (unlike FAST), and without lower bounds (unlike Rafiei and Keogh et al.). The price is that candidate selection becomes approximate, which those exact methods avoid. The contribution we can support with evidence is therefore narrower than a new algorithm: (1) a measured diagnosis that zero-lag candidates fail for this scorer because of their metric, not because of approximate search; (2) an ablation separating grid sparsity from index error, with an explanation of the grid spacing from envelope smoothness; (3) a held-out confirmation; and (4) an integration that keeps one contract on both paths, with measured costs of storage, filters and maintenance. Implementing the method in PostgreSQL is not by itself a contribution; keeping intervals, filters, row-level security and commit-time visibility identical on the exact and approximate paths is the part we evaluate.

The single-traversal alternative. Rafiei and Keogh et al. show that processing all transformations in one exact traversal with lower bounds can beat one search per transformation. We tested the two bounds closest to this design on our scorer before building probe search [32]: a Keogh-style envelope bound and a spectral bound, each covering all lags $|\ell| \le L$, on 1,826 windows of 2022 data. Even with the true tenth-best distance as threshold and a free bound, their combination pruned 63% of candidates (median overestimate of the best correlation 1.83×), which caps an exact pruned scan at about 1.9× on that gallery and 2.3× in the limit of large galleries, against the 5.4× of probes over the fastest exact path. Finer wedges, one bound per group of neighbouring lags, would be tighter; we did not test them. Smooth, normalized seismic envelopes have similar spectra and running ranges, which is the information such bounds keep.

12 LIMITATIONS

One signal domain. All retrieval results use seismic envelopes from one network. The method needs only a bounded-lag inner-product scorer, but how many probes and candidates suffice depends on envelope smoothness and on how often best lags

fall between grid points; we have not measured this on vibration, acoustic or physiological data. The title-level claim about *waveforms* therefore rests on one domain.

One implementation state per study. Development, confirmation, product checks, filtered search and load runs were measured on successive implementation states over several weeks: exact search became a parallel statement, routing and filter-aware probes were added, and envelope building was batched. Each study compares arms within one state, but cross-study ratios (for example, product probe latency against an exact path from another study) mix states. A consolidated rerun on one frozen commit has not been made.

Measurement conditions. One laptop-class machine, warm caches, pgvector's HNSW in one configuration and two budgets for zero-lag shortlists. Later runs shared the machine with an unrelated CPU-intensive job; we report its load average but cannot remove its effect. Load tests ran 15 minutes; a two-hour run was pending at the time of writing, and we make no claim about longer operation, archives beyond about one million windows, cold caches or several tenants under load.

What recall means. Recall@10 measures fidelity to the scorer D. Whether the bounded-lag top ten are the intervals an analyst wants is a separate question; we have a catalogue-distance proxy for seismic events in the supplement, but no task evaluation of probe search.

Approximation without bounds. Probe search can miss a window that the exact path returns (per query, down to 7 of 10 on held-out data). The router removes this risk only where it chooses the exact path. Exact pruning with lag-uniform lower bounds was capped near 2× on our data (Section 11); finer, per-lag-group bounds were not tested.

Grid choice. The 13-probe grid was chosen on dev data and explained post hoc by envelope smoothness on dev data; it was confirmed on held-out data but only for this representation.

13 CONCLUSION

Bounded-lag waveform search can use a standard inner-product index if the index is asked the right question. A zero-lag shortlist asks a different one, and lost a quarter of the scorer's top ten whether it was computed exactly or approximately. Because each lag term is an inner product with a shifted query, thirteen shifted probes into one unmodified HNSW index, re-ranked with the unchanged scorer, recovered 97.6% of the top ten on held-out stations and year at about a fifth of the latency of the fastest exact path; an exact-candidate control showed that nearly all of the remaining loss comes from the index, not from the sparse grid. Inside PostgreSQL, the same contract holds on both paths: hits are sample intervals of losslessly stored recordings, filters and row-level security apply, and new windows are searchable at commit. The benefit is bounded by three costs: an index copy of 1.0–1.4× the raw data, a router that must send selective filters to exact search, and write-ahead logging from index maintenance that default checkpoint settings do not contain. A consolidated comparison on one frozen implementation, and a second signal domain, are the evidence still missing for a general claim.

REFERENCES

[1] pgvector contributors. pgvector: open-source vector similarity search for Postgres, version 0.8.0. https://github.com/pgvector/pgvector.

[2] Clara E. Yoon, Ossian O'Reilly, Karianne J. Bergen, and Gregory C. Beroza. 2015. Earthquake detection through computationally efficient similarity search. *Science Advances* 1, 11, e1501057. doi:10.1126/sciadv.1501057.

[3] Thanawin Rakthanmanon, Bilson Campana, Abdullah Mueen, Gustavo Batista, Brandon Westover, Qiang Zhu, Jesin Zakaria, and Eamonn Keogh. 2012. Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping. In *Proc. 18th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD '12)*, 262–270. doi:10.1145/2339530.2339576.

[4] GFZ German Research Centre for Geosciences and Institut des Sciences de l'Univers–Centre National de la Recherche CNRS-INSU. 2006. IPOC Seismic Network. Integrated Plate boundary Observatory Chile. Network code CX. doi:10.14470/PK615318.

[5] MorletDB contributors. 2026. MorletDB source repository, protocols and results. https://github.com/merkhet-io/morletdb.

[6] MorletDB contributors. R1b confirmation (2022). `evaluation/materialized/compact/results/conf-2022/RESULTS.md` in [5].

[7] MorletDB contributors. R2: retrieval scaling, and R2-D diagnostic. `evaluation/retrieval_scaling/` in [5].

[8] John Paparrizos and Luis Gravano. 2015. k-Shape: Efficient and Accurate Clustering of Time Series. In *Proc. 2015 ACM SIGMOD Int. Conf. on Management of Data (SIGMOD '15)*. doi:10.1145/2723372.2737793.

[9] Abdullah Mueen et al. The Fastest Similarity Search Algorithm for Time Series Subsequences under Euclidean Distance. https://www.cs.unm.edu/~mueen/FastestSimilaritySearch.html.

[10] MorletDB contributors. BL1: shifted-probe retrieval, confirmation and ingestion. `evaluation/bounded_lag/` in [5].

[11] MorletDB contributors. SS1: one stored envelope representation. `evaluation/search_storage/SS1_RESULTS.md` in [5].

[12] MorletDB contributors. SS3: filtered envelope search. `evaluation/search_storage/SS3_RESULTS.md` in [5].

[13] MorletDB contributors. Pilot stages 1–2: access control, recovery and load. `evaluation/pilot/RESULTS.md` in [5].

[14] Davood Rafiei. 1999. On Similarity-Based Queries for Time Series Data. In *Proc. 15th Int. Conf. on Data Engineering (ICDE)*.

[15] Eamonn Keogh, Li Wei, Xiaopeng Xi, Sang-Hee Lee, and Michail Vlachos. 2006. LB_Keogh Supports Exact Indexing of Shapes under Rotation Invariance with Arbitrary Representations and Distance Measures. In *Proc. 32nd Int. Conf. on Very Large Data Bases*, 882–893. Extended in *The VLDB Journal* 18 (2009), 611–630. doi:10.1007/s00778-008-0111-4.

[16] Michail Vlachos, Zografoula Vagena, Philip S. Yu, and Vassilis Athitsos. 2005. Rotation Invariant Indexing of Shapes and Line Drawings. In *Proc. 14th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 131–138. doi:10.1145/1099554.1099580.

[17] Leslie R. Foulds, Jorge P. de Morais Neto, Humberto J. Longo, Hugo A. D. do Nascimento, and Wellington S. Martins. 2015. A variant of k-nearest neighbors search with cyclically permuted query points for rotation-invariant image processing. *Discrete Applied Mathematics* 197, 123–144. doi:10.1016/j.dam.2015.02.011.

[18] John Paparrizos and Luis Gravano. 2017. Fast and Accurate Time-Series Clustering. *ACM Trans. Database Syst.* 42, 2, Article 8. doi:10.1145/3044711.

[19] John Paparrizos and Michael J. Franklin. 2019. GRAIL: Efficient Time-Series Representation Learning. *Proc. VLDB Endow.* 12, 11, 1762–1777. doi:10.14778/3342263.3342648.

[20] Han Qiu, Hoang Thanh Lam, Francesco Fusco, and Mathieu Sinn. 2018. Learning Correlation Space for Time Series. arXiv:1802.03628.

[21] Kexin Rong, Clara E. Yoon, Karianne J. Bergen, Hashem Elezabi, Peter Bailis, Philip Levis, and Gregory C. Beroza. 2018. Locality-Sensitive Hashing for Earthquake Detection: A Case Study of Scaling Data-Driven Science. *Proc. VLDB Endow.* 11, 11, 1674–1687.

[22] Rigobert Tibi, Christopher Young, Antonio Gonzales, Sanford Ballard, and Andre Encarnacao. 2017. Rapid and Robust Cross-Correlation-Based Seismic Signal Identification Using an Approximate Nearest Neighbor Method. *Bull. Seismol. Soc. Am.* 107, 4, 1954–1968. doi:10.1785/0120170011.

[23] Antonio Gonzales, Nicholas Blazier, et al. Seismic Waveform Matching via Approximate Nearest Neighbor with Projection Deficiency Compensation. Sandia National Laboratories, OSTI 1458306.

[24] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proc. 43rd Int. ACM SIGIR Conf.*, 39–48. doi:10.1145/3397271.3401075.

[25] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proc. VLDB Endow.* 13, 12, 3152–3165.

[26] Jianguo Wang et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proc. 2021 Int. Conf. on Management of Data (SIGMOD)*. doi:10.1145/3448016.3457550.

[27] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 2 (SIGMOD 2024). doi:10.1145/3654923.

[28] Qianxi Zhang et al. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*.

[29] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, et al. 2024. SingleStore-V: An Integrated Vector Database System in SingleStore. *Proc. VLDB Endow.* 17, 12.

[30] Jiaye Wu, Peng Wang, Ningting Pan, Chen Wang, Wei Wang, and Jianmin Wang. 2019. KV-Match: A Subsequence Matching Approach Supporting Normalization and Time Warping. In *Proc. 35th IEEE Int. Conf. on Data Engineering (ICDE)*. arXiv:1710.00560.

[31] Michele Linardi and Themis Palpanas. 2018. Scalable, Variable-Length Similarity Search in Data Series: The ULISSE Approach. *Proc. VLDB Endow.* 11, 13, 2236–2248. doi:10.14778/3275366.3275372.

[32] MorletDB contributors. Certified pruning for envelope search: headroom and feasibility. `evaluation/pruning/README.md` in [5].

[33] MorletDB contributors. Grid-spacing analysis on development data. `research/paper/analysis/` in [5].

A CLAIMS AND EVIDENCE

Each claim of the paper with the study that supports it, the measurement kind (E: experiment, P: product, H: historic) and data (D: dev, C: held-out), and its main restriction. Full protocols and results: [5].

Claim	Study	Restriction
zero-lag top 1000 + re-rank: recall 0.743 (exact), 0.744 (HNSW)	R2-D; E, D	two budgets, one HNSW configuration
HNSW and exact shortlists share 98–100% of candidates	R2-D; E, D	largest gallery
13-lag grid exact = all 61 lags (1.000)	BL1; E, D	dev only; offline control
7-lag grid loses 0.014	BL1; E, D	dev only
spacing 5 keeps $\geq$0.944 envelope autocorrelation (p5)	grid; E, D	post hoc; one representation
lag-uniform bounds prune $\leq$63%, $\leq$2.3×	pruning; E	2022 windows; oracle threshold
probes 0.976 (min 0.7; 5/100 <0.9)	BL1-C; E, C	seismic; unfiltered
index approximation costs 0.023	BL1-C; E, C	includes float16 probes
same-count zero-lag: 0.613	BL1-C; E, C	descriptive control
p95 131 vs 713 ms (fastest exact)	BL1-C; E, C	one machine, warm
product function: 0.977, p95 50 ms	product check; P, C	later implementation

Claim	Study	Restriction
post-filter 0.515 at 1.8% eligible; routed 1.0	SS3; E/P, C	five filter levels
routed recall 0.977–1.0, no filter or RLS violations	SS3; P, D+C	latency bound missed by 0.8 ms
index structures 1.00–1.39× raw	BL1, BL1-C; E	depends on raw compressibility
float16 envelopes: recall 0.9999	R1b; E	2022 event windows
one stored copy: −250 MB, +23% or +27–32% p95	SS1; E, D+C	both missed latency bounds
searchable at commit; 919k samples/s	BL1-S; E, D	one day, four channels
409.6k samples/s for 15 min, capacity 1.3–1.4×	C2, C3; P	15-min runs; shared machine
WAL 13.8 → 1.91 GB per 15 min	C2, C2p; P	four settings changed together; load 11.7 vs 7.3
companion-table RLS: exact +5%	Stage 1; P	one tenant layout